# PUBLISHING A QUARTO SITE TO GITHUB PAGES

*A step-by-step guide, from first setup to routine updates*

This guide walks through getting a Quarto-built website live on GitHub Pages, and the routine you'll repeat every time you update it afterward. It assumes no prior GitHub experience.

A visual key before you start: `text like this` is something you type exactly as shown. A colored tag like the ones below tells you *where* to type it.

*Anything in angle brackets, like <your-username>, is a placeholder — replace it with your own information, angle brackets and all removed.*

**EXAMPLE TAG**

This means the instruction happens in a specific place — Terminal, your browser, or an editor. Look for these before every code block.

## Part One — Initial Setup (one time only)

Do this section once, when you're first putting a project online. Prerequisites: Quarto installed, a GitHub account, and RStudio (or another editor – VS Code, Positron) with a working Quarto project on your machine.

### 1. Create your GitHub repository (and an account)

**BROWSER — GITHUB.COM**

First, if you haven’t done so already, create a GitHub account. Keep your username as short as possible while being appropriate to your name and brand – brevity is better for your website’s URL.

Once that’s done, create a new repository (“repo”). The standard root address is <your-username>.github.io, so name the repository exactly that — GitHub treats that name as special and serves it at the root domain. Any other name publishes as a project site, nested at <your-username>.github.io/<repo-name>/ instead.

Make sure the repository is set to Public. Free GitHub Pages hosting only publishes automatically from public repositories — private-repo Pages hosting requires a paid GitHub plan. This means anyone can look through the root files, so don’t upload any state secrets among your files.

### 2. Point your project at that address

**RSTUDIO EDITOR — _QUARTO.YML**

In your project's _quarto.yml file, set site-url to match where the site will actually live:

```
site-url: "https://<your-username>.github.io/"
```

This doesn't change how your site looks — it's used for things like the sitemap and social-preview links, so it should match reality.

## 3. Set your git identity

**TERMINAL**

Git labels every saved change with an author. This is a one-time setting per computer, not per project:

```
git config --global user.name "<Your Name>"
git config --global user.email "<your-email@example.com>"
```

Use the email address tied to your GitHub account. Your name can be anything, but if your real name is linked to your GitHub account, it's easiest to just use that. If you use multiple computers for this, you may want to specify that in your `user.name` though.

## 4. Authenticate with GitHub

**TERMINAL, THEN BROWSER**

GitHub no longer accepts your account password for pushing code from the command line. The simplest fix is installing the GitHub CLI and running:

```
gh auth login
```

This opens a browser window to confirm the login and stores your credentials — you won't need to repeat this per project. Skipping this step is the most common reason a later push fails with an authentication error.

Note that I (Michael) opened a separate Mac Terminal window to do this – I used the built-in R Terminal / Command Line for everything else. I did this to make sure that GitHub CLI installs outside of the Quarto directory, which is where the R-based terminal is rooted.

## 5. Connect your local project to the GitHub repo

**TERMINAL, INSIDE YOUR PROJECT FOLDER**

Turn the folder into a git repository (skip this if it already is one):

```
git init
```

Then tell git where the cloud copy lives, aliased as "origin":

```
git remote add origin https://github.com/<your-username>/<repo-name>.git

# Ex. git remote add origin https://github.com/mbvargo/mbvargo.github.io.git
```

## 6. Commit and push your source files

**TERMINAL**

```
git add .
git commit -m "Initial commit"
git push -u origin main
```

This uploads your source files (.qmd, .scss, etc.) to the main branch — think of it as backing up the raw ingredients of your site, separate from the rendered result. For the quoted text in the second line (`git commit -m "Initial commit"`), the text value determines the message attached to that upload of your files. So for subsequent pushes, you could list relevant changes to the files (e.g., `git commit -m "Added blog page and 3 new projects"`)

## 7. Publish the rendered site

TERMINAL

```
quarto publish gh-pages
```

This is the step that actually builds the HTML from your .qmd files and pushes it to a second, separate branch called gh-pages — the branch GitHub Pages actually serves to visitors. The first run will open a browser tab to confirm the publish.

## 8. Verify Pages is enabled

BROWSER — GITHUB.COM

***Critical***: In your repository, go to Settings → Pages and confirm the source is set to "Deploy from a branch" with gh-pages selected (quarto publish usually sets this automatically). ***Give it a minute***, then visit your site's address. If it doesn't look right, check the following link:

https://github.com/<user.name>/<user.name>.github.io/actions.

This will show the success or failure of your pushes, and you can also find any error codes associated with the push. You'll know if there's something wrong by the icon next to your workflow runs – a red X will appear for failures, a green checkmark appears for successful runs.

# Part Two — Updating Your Site (every time)

Once setup is done, updating the live site after making edits is a short, repeatable routine. Do this from inside your project folder.

### 1. Preview your changes

TERMINAL, OR RSTUDIO'S RENDER BUTTON

```
quarto preview
```

Confirms your edits look right locally. Nothing leaves your machine at this step.

### 2. Check what changed

TERMINAL

```
git status
```

Lists every file you've modified, added, or deleted since your last commit — a quick check before saving.

### 3. Stage and commit

TERMINAL

```
git add .
git commit -m "<short description of what changed>"
```

Staging and committing are separate steps on purpose: add marks files as ready, commit saves that labeled snapshot, building a real history of what changed and when.

### 4. Push the source

TERMINAL

```
git push
```

Backs up your source files to GitHub's main branch. This alone does not update the live site.

### 5. Publish the update

TERMINAL

```
quarto publish gh-pages
```

Rebuilds the HTML and pushes it to gh-pages — this is the step that actually updates what visitors see.

### 6. Check the live site

BROWSER

Reload your site's address (a hard refresh — Ctrl/Cmd+Shift+R — helps if your browser cached the old version) to confirm the update went through.

# Quick Reference

Once you're comfortable, this is the whole routine in order, no explanation needed:

```
quarto preview
git status
git add .
```

```
git commit -m "<message>"
git push
quarto publish gh-pages
```

## Common Hiccups

- "Repository not found" or an authentication error on push — usually means step 4 (GitHub authentication) wasn't completed, or the remote URL in step 5 has a typo.
- Site shows a 404 — check Settings → Pages is pointed at the gh-pages branch, and that the repository is Public, not Private.
- git commit says "nothing to commit" — this is normal if quarto preview didn't change any tracked source files; it usually means there was nothing new to save.
- Site updated on GitHub but looks unchanged in the browser — try a hard refresh (Ctrl/Cmd+Shift+R); browsers cache pages aggressively.